

Лабораторна робота №2

Тема: ДОСЛІДЖЕННЯ ТА РЕАЛІЗАЦІЯ МЕТОДУ РІЗНИЦІ ЗНАЧЕНЬ ПІКСЕЛІВ (PVD) ДЛЯ ПРИХОВУВАННЯ ІНФОРМАЦІЇ У ЗОБРАЖЕННЯХ

Мета: ознайомитися з методом Pixel Value Differencing (PVD); реалізовувати спрощений алгоритм PVD для вбудовування бітового повідомлення у два піксельні значення та вилучення вбудованих бітів

Хід роботи:

Завдання 1. Реалізовувати спрощений алгоритм PVD в середовищі MS Excel або Google Таблиці (на додаткові бали – на будь-якій мові програмування) для вбудовування бітового повідомлення у два піксельні значення та вилучення вбудованих бітів. На мові програмування також може бути повноцінна реалізація PVD для зображення у градації сірого.

Було реалізовано програму на мові Python.

Лістинг:

```
import math
from PIL import Image
import os

RANGE_TABLE = [
    (0, 7), (8, 15), (16, 31),
    (32, 63), (64, 127), (128, 255)
]

INPUT_DIR = "download"
OUTPUT_DIR = "result"

def get_interval(diff):
    abs_diff = abs(diff)
    for low, high in RANGE_TABLE:
        if low <= abs_diff <= high:
            return low, high
    return None, None
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.2			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Незнайко В.К.			Звіт з лабораторної роботи №2		Лім.	Арк.
Перевір.		Щур Н.О.						1
Керівник							ФІКТ Гр. ВТк-24-1	
Н. контр.								
Зав. каф.		Морозов А.В.						

```

def is_png_file(path):
    return path.lower().endswith(".png")

def clamp_value(val):
    return max(0, min(255, val))

def compute_capacity(image):
    pixels = image.load()
    width, height = image.size

    capacity_bits = 0

    for y in range(height):
        for x in range(0, width - 1, 2):
            _, _, b1 = pixels[x, y]
            _, _, b2 = pixels[x + 1, y]

            diff = b2 - b1
            low, high = get_interval(diff)

            if low is None:
                continue

            capacity_bits += int(math.log2(high - low + 1))

    return capacity_bits

def encode_image(input_path, output_path, message):
    if not is_png_file(input_path):
        print("Only PNG files are supported")
        return

    img = Image.open(input_path).convert("RGB")
    pixels = img.load()
    width, height = img.size

    binary_message = ''.join(format(ord(c), '08b') for c in message) + '00000000'

    capacity = compute_capacity(img)

    if len(binary_message) > capacity:
        print("Message is too large for this image")
        return

    index = 0

    for y in range(height):
        for x in range(0, width - 1, 2):

```

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.2	Арк.
		Щур Н.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

if index >= len(binary_message):
    img.save(output_path)
    print("Encoding finished")
    return

r1, g1, b1 = pixels[x, y]
r2, g2, b2 = pixels[x + 1, y]

diff = b2 - b1
low, high = get_interval(diff)

if low is None:
    continue

n_bits = int(math.log2(high - low + 1))

chunk = binary_message[index:index + n_bits].ljust(n_bits, '0')
value = int(chunk, 2)

new_diff = low + value
m = new_diff - abs(diff)

if diff >= 0:
    b1_new = b1 - m // 2
    b2_new = b2 + (m - m // 2)
else:
    b1_new = b1 + m // 2
    b2_new = b2 - (m - m // 2)

b1_new = clamp_value(b1_new)
b2_new = clamp_value(b2_new)

pixels[x, y] = (r1, g1, b1_new)
pixels[x + 1, y] = (r2, g2, b2_new)

index += n_bits

img.save(output_path)
print("File saved\n")

def decode_image(input_path):
    if not is_png_file(input_path):
        print("Only PNG files are supported")
        return

    img = Image.open(input_path).convert("RGB")
    pixels = img.load()
    width, height = img.size

    bitstream = ""

```

```

for y in range(height):
    for x in range(0, width - 1, 2):

        _, _, b1 = pixels[x, y]
        _, _, b2 = pixels[x + 1, y]

        diff = b2 - b1
        low, high = get_interval(diff)

        if low is None:
            continue

        n_bits = int(math.log2(high - low + 1))
        value = abs(diff) - low

        bitstream += format(value, f'0{n_bits}b')

message = ""

for i in range(0, len(bitstream), 8):
    byte = bitstream[i:i + 8]
    if byte == "00000000":
        break
    message += chr(int(byte, 2))

print(message)

if __name__ == "__main__":
    os.makedirs(INPUT_DIR, exist_ok=True)
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    while True:
        mode = input("Select mode:\n1 - Encode\n2 - Decode\n0 - Exit\n> ")

        if mode == "1":
            file_name = input("Input file name: ")
            text = input("Message: ")

            encode_image(
                os.path.join(INPUT_DIR, file_name),
                os.path.join(OUTPUT_DIR, "encoded_" + file_name),
                text
            )

        elif mode == "2":
            file_name = input("File name: ")

            decode_image(
                os.path.join(OUTPUT_DIR, file_name)

```

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.2	Арк.
		Щур Н.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		4

```

    )
else:
    break;

```

Результат виконання:

```

Select mode:
1 - Encode
2 - Decode
0 - Exit
> 1
Input file name: SpongeBob.png
Message: Vitalik Neznyako
Encoding finished
Select mode:
1 - Encode
2 - Decode
0 - Exit
> 2
File name: encoded_SpongeBob.png
Vitalik Neznyako
Select mode:
1 - Encode
2 - Decode
0 - Exit
> 

```

Рис. 2.1. Результат виконання



Рис. 2.2. Оригінальне і зашифроване зображення

Також було проведено тестування.

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.2	Арк.
		Щур Н.О.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

ЛІСТИНГ:

```
import unittest
from pvd import clamp_value, get_interval
import math

class TestPVD(unittest.TestCase):

    def test_get_interval(self):
        print("\n=== TEST get_interval ===")

        cases = [5, 10, 20, 50, 100, 200]

        for d in cases:
            result = get_interval(d)
            print(f"d={d} -> interval={result}")
            self.assertIsNotNone(result)

    def test_clamp(self):
        print("\n=== TEST clamp ===")

        cases = [-10, 300, 128]

        for v in cases:
            result = clamp_value(v)
            print(f"clamp({v}) -> {result}")
            self.assertTrue(0 <= result <= 255)

    def test_n_bits(self):
        print("\n=== TEST n_bits ===")

        ranges = [(0,7), (8,15), (16,31), (32,63), (64,127), (128,255)]

        for l, u in ranges:
            n = int(math.log2(u - l + 1))
            print(f"range=({l},{u}) -> n={n}")
            self.assertGreater(n, 0)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Результат тестування:

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.2	Арк.
		Щур Н.О.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

=== TEST clamp ===
clamp(-10) -> 0
clamp(300) -> 255
clamp(128) -> 128
.
=== TEST get_interval ===
d=5 -> interval=(0, 7)
d=10 -> interval=(8, 15)
d=20 -> interval=(16, 31)
d=50 -> interval=(32, 63)
d=100 -> interval=(64, 127)
d=200 -> interval=(128, 255)
.
=== TEST n_bits ===
range=(0,7) -> n=3
range=(8,15) -> n=3
range=(16,31) -> n=4
range=(32,63) -> n=5
range=(64,127) -> n=6
range=(128,255) -> n=7
.
-----
Ran 3 tests in 0.001s

OK

```

Рис. 2.3. Результати тестування

Висновок: В ході виконання лабораторної роботи ознайомилися з методом Pixel Value Differencing (PVD); реалізовувати спрощений алгоритм PVD для вбудовування бітового повідомлення у два піксельні значення та вилучення вбудованих бітів

		Незнайко В.К.			ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.07.000 – Лр.2	Арк.
		Щур Н.О.				7
Змн.	Арк.	№ докум.	Підпис	Дата		